Chương 12: Mô hình tùy chỉnh và Huấn luyện với TensorFlow

Giảng viên: TS. Trần Thành Thắng

Đại học Đông Á

Năm học 2024-2025

Nội dung chương

1. Giới thiệu TensorFlow & Kiến trúc
2. Sử dụng TensorFlow như NumPy
3. Tùy chỉnh mô hình và Thành phần (Custom Components)
4. Hàm và biểu đồ TensorFlow (AutoGraph)
5. Vòng lặp Huấn luyện Tùy chỉnh

Giới thiệu nhanh về TensorFlow

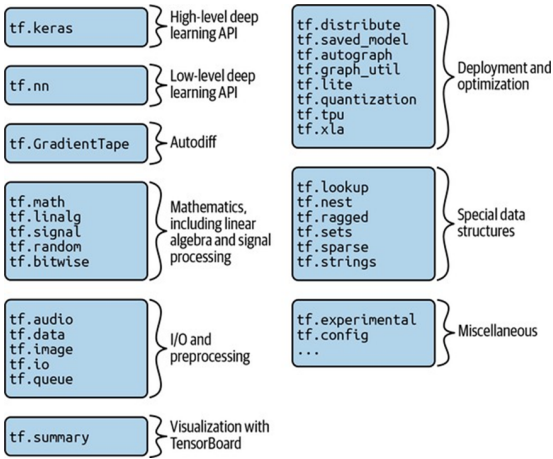
- TensorFlow là thư viện tính toán số học mãnh mẽ mã nguồn mở, đặc biệt phù hợp cho học máy quy mô lớn và học sâu.
- Cốt lõi của nó tương tự NumPy, nhưng thao tác với **Tensor** thay vì mảng.
- Cung cấp khả năng tính toán vượt trội nhờ hỗ trợ phần cứng tăng tốc: **GPU** (Graphics Processing Unit) và **TPU** (Tensor Processing Unit).
- Tính năng cốt lõi: Tính toán phân tán trên hàng ngàn thiết bị và server.
- Có thể biên dịch tự động các biểu thức tính toán Python thành **Biểu đồ (Graph)** C++ hiệu suất cao thông qua công nghệ AutoGraph.

Hệ sinh thái TensorFlow

- **TensorBoard**: Trực quan hóa dữ liệu, đồ thị mô hình và phân tích quá trình huấn luyện.
- **TensorFlow Hub**: Nơi tải xuống và sử dụng lại các mô hình đã huấn luyện (Pretrained models) giúp đẩy nhanh tiến độ dự án.
- **TensorFlow Extended (TFX)**: Bộ công cụ chuyên biệt để xây dựng quy trình ML toàn diện và triển khai mô hình lên môi trường sản xuất (Production).
- **TF Lite & TF.js**: Hỗ trợ triển khai mô hình học máy trên thiết bị di động (Android, iOS), phần cứng nhúng (Raspberry Pi) và ngay trên trình duyệt Web bằng JavaScript.

Tổng quan về API Python của TensorFlow (1/2)

- API Python của TensorFlow được thiết kế phân tầng từ thấp đến cao.
- Ở mức thấp, ta có các API nền tảng: các phép toán trên Tensor, biến số (Variables), tự động vi phân (tf.GradientTape).
- Ở cấp độ cao hơn (Cấp cao), Keras cung cấp API trực quan (tf.keras) để xây dựng lớp (layers) và mô hình (models).
- API dữ liệu (tf.data) cung cấp quy trình nạp, xử lý trước và cung cấp dữ liệu huấn luyện hiệu quả.



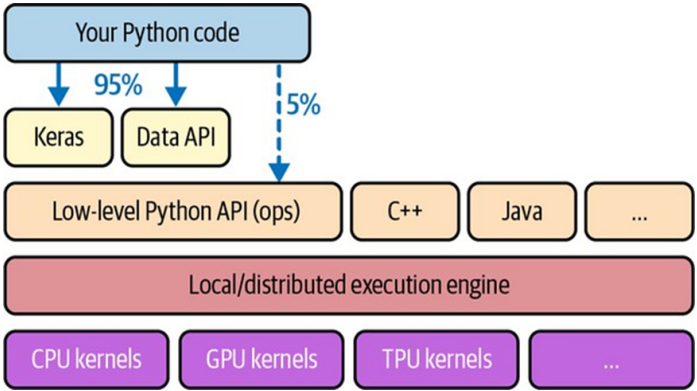
Hình 12-1: Tổng quan về API Python của TensorFlow

Tổng quan về API Python của TensorFlow (2/2)

- Sự trừu tượng hóa các mức độ phức tạp giúp người học mới tiếp cận qua Keras một cách nhanh chóng.
- Trong khi đó, các nhà nghiên cứu sâu (researchers) vẫn có thể dùng các API cấp thấp để viết các thuật toán huấn luyện phi tiêu chuẩn (như trong GANs hay Meta-learning).
- Hình 12-1 tóm tắt một cách bao quát các module quan trọng mà TensorFlow cung cấp.
- Chúng ta sẽ lần lượt khám phá từ Tensor cơ bản đến việc tùy chỉnh các lớp và mô hình thông qua API này.

Kiến trúc của TensorFlow (1/2)

- Hình 12-2 minh họa kiến trúc bên trong của TensorFlow.
- Lớp Execution Engine ở trung tâm thực hiện các biểu đồ (Graphs) bằng C++ (mã nguồn lõi được tối ưu cực cao).
- Cung cấp khả năng chạy đồng thời (multi-threading) trên nhiều loại phần cứng (CPU, GPU, TPU) và qua mạng (Android, iOS, Cloud).



Hình 12-2: Kiến trúc của TensorFlow

Kiến trúc của TensorFlow (2/2)

- API giao diện bằng nhiều ngôn ngữ: Python (phổ biến nhất), C++, Java, Go, Swift.
- Hầu hết thời gian bạn sẽ chỉ cần tương tác qua Keras (High-level API).
- Lớp **XLA (Accelerated Linear Algebra)**: Trình biên dịch mạnh mẽ, giúp tối ưu hóa thêm các Graph, kết hợp các toán tử lại (kernel fusion) nhằm tăng tốc tối đa tốc độ xử lý phần cứng.
- TF Distributed Execution: Trực tiếp phân phối khối lượng tính toán trên nhiều máy vật lý (Parameter Server, MirroredStrategy).

Tensor và Phép toán (Tensors and Operations)

- API của TensorFlow chủ yếu xoay quanh các *tensors* (cáp số liệu đa chiều).
- Khái niệm Tensors được kế thừa từ Toán học, dùng để đại diện cho đại lượng vô hướng (scalar - 0D), vectơ (1D), ma trận (2D) hoặc không gian đa chiều (nD).
- Cách dễ nhất để tạo tensor là dùng `tf.constant()`.

```
import tensorflow as tf

# Tao tensor tu ma tran 2 chieu (2D)
t = tf.constant([[1., 2., 3.], [4., 5., 6.]])
print(t.shape) # In ra: TensorShape([2, 3])
print(t.dtype) # In ra: <dtype: 'float32'>
```

Các hàm toán học cơ bản

- TensorFlow cung cấp hàng loạt hàm toán học mạnh mẽ hoạt động trực tiếp trên tensors mà không cần vòng lặp for.
- Các toán tử chuẩn Python (+, −, *, /, **) đã được nạp chồng (overloaded).

```
# Tinh toan co ban bang TF operations
t1 = t + 10                    # Phep cong (hoat dong broadcasting)
t2 = tf.square(t)              # Binh phuong tung phan tu (element-wise)
t3 = t @ tf.transpose(t)       # Nhan ma tran (Matrix multiplication)
```

- Ký hiệu `@` xuất hiện từ Python 3.5 để thay thế cho phép nhân ma trận (`tf.matmul`).
- Các hàm phổ biến khác: `tf.exp()`, `tf.sqrt()`, `tf.reduce_sum()`, `tf.reduce_mean()`.

Tensors và NumPy: Tương tác qua lại

- Tensors tương tác rất thân thiện và liền mạch với thư viện NumPy - tiêu chuẩn của khoa học dữ liệu.
- Có thể tạo tensor trực tiếp từ một mảng NumPy, và ngược lại.

```
import numpy as np

# Chuyen Numpy -> Tensor
a = np.array([2., 4., 5.])
t_a = tf.constant(a)

# Chuyen Tensor -> Numpy
a_back = t_a.numpy()        # Ban co thu dung np.array(t_a)

# Tinh toan ket hop (Cross-computation)
t_b = tf.square(a)          # TF chap nhan Numpy array
a_b = np.square(t_a)        # Numpy chap nhan TF Tensor
```

Lưu ý khi dùng chung Numpy và TensorFlow

- Mặc dù chúng có thể được truyền vào hàm của nhau, nhưng bạn nên ưu tiên dùng hàm TensorFlow nếu muốn tận dụng tốc độ của GPU.
- NumPy chỉ chạy được duy nhất trên CPU. Nếu một Tensor đang nằm trên GPU mà bạn gọi hàm NumPy, TensorFlow sẽ phải **sao chép dữ liệu từ bộ nhớ GPU về CPU** $\rightarrow$ Chi phí truyền tải lớn.
- Khác biệt nhỏ về kiểu dữ liệu: NumPy mặc định dùng float64 cho độ chính xác cao. TensorFlow mặc định dùng float32 (thậm chí bfloat16) vì mạng Neural quan trọng tính hiệu suất.

Chuyển đổi kiểu dữ liệu (Type Conversions)

- TensorFlow **không tự động** ép kiểu (type conversion) giống như Python hay NumPy.
- Việc thực hiện ép kiểu giữa `float32` và `float64` hoặc `float32` và `int32` có chi phí tính toán cao.

```
# Kieu int32 khong the cong voi float32 ngam dinh:
# tf.constant(2.0) + tf.constant(40) # --> Loi TypeError

# Can phai ep kieu chu dong va ro rang bang tf.cast:
t2 = tf.constant(40., dtype=tf.float64)

# tf.cast(t2, tf.float32) ha do chinh xac xuong float32
result = tf.constant(2.0) + tf.cast(t2, tf.float32)
```

Biến số (Variables)

- Đặc tính của `tf.constant()` là không thể thay đổi (immutable). Ta không thể gán lại một giá trị cho một phần tử bên trong nó.
- Tuy nhiên, mạng nơ-ron liên tục cập nhật **trọng số (weights)** và độ lệch (biases) qua mỗi chu kỳ học (epoch).
- Do đó, ta cần sử dụng cấu trúc `tf.Variable`.

```
v = tf.Variable([[1., 2., 3.], [4., 5., 6.]])

# Cap nhat toan bo bien (dung phuong thuc assign)
v.assign(2 * v)

# Cap nhat vao dung 1 vi tri cu the (nhu numpy arrays)
v[0, 1].assign(42)

# Cap nhat toan cuc bang phep cong (+), tru (-)
v.assign_add([[1., 1., 1.], [1., 1., 1.]])
```

Các cấu trúc dữ liệu nâng cao khác

- **Sparse tensors (`tf.SparseTensor`):** Tối ưu bộ nhớ cho ma trận chứa phần lớn giá trị là 0. Lưu trữ 3 mảng: *indices*, *values*, *dense_shape*.
- **Tensor arrays (`tf.TensorArray`):** Một danh sách các Tensors cho phép append động. Rất hữu ích cho các mạng tuần tự RNN.
- **Ragged tensors (`tf.RaggedTensor`):** Đại diện mảng bất đối xứng. VD: danh sách chứa các chuỗi con có độ dài không bằng nhau.
- **String tensors:** Tensor lưu trữ kiểu chữ (strings), các chuỗi ký tự UTF-8 hoặc byte arrays, cần thiết cho NLP.
- **Sets:** Tập hợp Toán học dưới dạng Sparse tensor bằng `tf.sets`.